Probeklausur: Echtzeitsysteme (EZS)

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Allgemeine Hinweise

- **Zweck:** Vorbereitung auf die Papierklausur
- **Bearbeitungszeit:** 90 Minuten
- **Hilfsmittel:** Keine (Closed Book)
- **Maximale Punktzahl:** 100 Punkte

Alle Angaben sind Lehrannahmen. Hardware und Programmierwerkzeuge werden nicht benötigt. Zeigen Sie Rechenwege und begründen Sie Aussagen. API-Namen im Code sind vorgegeben; beurteilt wird ihr Einsatz, nicht die Erinnerung an Werkzeugmenüs.

Aufgabe 1: Scheduling-Theorie & RMS (25 Punkte)

Ein neues Steuergerät soll drei periodische Tasks im präemptiven Multitasking verarbeiten. Für die Tasks wurden folgende Parameter (Worst-Case Execution Time C_i, Periode T_i) ermittelt. Es gelten implizite Deadlines ($D_i = T_i$). Das Lehrmodell verwendet genau einen Prozessor, unabhängige periodische Aufgaben, vollständige Präemption und keine zusätzlichen Interrupt-, Wechsel- oder Sperrkosten. Die angegebenen C_i sind für dieses Modell als sichere obere Schranken vorgegeben, keine aus einer endlichen Messreihe garantierten Werte.

- τ_1: $C_1 = 2\,\text{ms}$, $T_1 = 8\,\text{ms}$
- τ_2: $C_2 = 3\,\text{ms}$, $T_2 = 12\,\text{ms}$
- τ_3: $C_3 = 4\,\text{ms}$, $T_3 = 24\,\text{ms}$

1.1 Berechnen Sie die Gesamt-CPU-Auslastung U des Systems. Zeigen Sie Ihren Rechenweg. *(5 Punkte)*

1.2 Vergeben Sie die Prioritäten der Tasks nach den Regeln des *Rate Monotonic Scheduling (RMS)*. Begründen Sie Ihre Wahl in einem Satz. *(5 Punkte)*

1.3 Prüfen Sie mithilfe der Liu-und-Layland-Schranke, ob dieses Task-Set garantiert schedulbar ist. Die Schranke für $n = 3$ lautet $\approx 78{,}0\,\%$. *(5 Punkte)*

1.4 Nennen Sie zwei Voraussetzungen des hier benutzten Liu-und-Layland-Tests. Erläutern Sie zu jeder einen konkreten Systemfall, in dem die einfache Rechnung unverändert nicht genügt. *(10 Punkte)*

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 2: Synchronisation & Priority Inversion (20 Punkte)

2.1 Erklären Sie den Unterschied zwischen einem CMSIS-RTOS2-Mutex und einem binären Semaphor im verwendeten FreeRTOS-Adapter. Gehen Sie auf *Ownership*, den Anfangswert eines Ereignissemaphors und die zulässige Verwendung aus einer ISR ein. ISR bedeutet Interrupt Service Routine. *(10 Punkte)*

2.2 Beschreiben Sie die indirekte Verzögerung von H bei $H > M > L$, wenn L einen von H benötigten Mutex hält und M ohne Ressourcenzugriff bereit wird. Wie hilft Prioritätsvererbung im Ein-Mutex-Fall? Welche H-Wartezeit bleibt? Beseitigt Vererbung einen Ressourcenwartezyklus? *(10 Punkte)*

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 3: Multicore & Architektur (20 Punkte)

Sie entwerfen ein System auf dem STM32H755 (Cortex-M7 und Cortex-M4).

3.1 Erklären Sie den Unterschied zwischen SMP (*Symmetric Multiprocessing*) und AMP (*Asymmetric Multiprocessing*). Ordnen Sie M7 Bare Metal und M4 mit lokalem FreeRTOS ein. Bietet diese Rollenaufteilung automatisch zeitliche Isolation? Nennen Sie zwei weiterhin gemeinsame Ressourcen. *(10 Punkte)*

3.2 Was ist ein *Cache Coherency Failure*? Beschreiben Sie, wie dieser Fehler auftritt, wenn der M7-Kern über Write-Back-Caches in den geteilten SRAM4 schreibt und der M4-Kern versucht, diese Daten zu lesen. Welche Rolle spielt eine passend konfigurierte nicht cachebare MPU-Region? Welche beiden zusätzlichen Verträge bleiben für geordnete und exklusive Veröffentlichung nötig? *(10 Punkte)*

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 4: Safety & Watchdogs (15 Punkte)

4.1 Im Lehrmodell wird ein IWDG ohne Fenster betrieben. Ein WWDG hat $f_{\mathrm{PCLK}} = 100\,\mathrm{MHz}$, Vorteiler $P = 8$, Reload $R = 127$ und Fenster $W = 80$. Erlaubt ist $63 < T \leq 80$, erster Abwärtsschritt ideal nach einem vollen Tick. Berechnen Sie mit $\Delta t = 4096P/f_{\mathrm{PCLK}}$ Tickdauer und Zeit bis Fensteröffnung. Beurteilen Sie einen Refresh nach 5 ms. Beweist ein Refresh im Fenster, dass alle Validierungsroutinen korrekt durchlaufen wurden? *(8 Punkte)*

4.2 Der WWDG fordert EWI bei $T = 64$ an und setzt beim nächsten Schritt nach 63 zurück. Erklären Sie Maskierbarkeit und nominale Restzeit. Im gegebenen Lehrvertrag muss ENABLE bei Fehler und während Reset/Boot inaktiv bleiben; der EWI soll kurz Low setzen und **nicht refreshen**. Warum darf sich der Ausgangsschutz trotzdem nicht allein auf diesen Callback verlassen? Nennen Sie eine Beschaltungs- und eine Bootmaßnahme für einen isolierten aktiven High-Ausgang. *(7 Punkte)*

Ihre Lösung

(Notieren Sie Rechnungen und Begründungen hier.)

Aufgabe 5: Code-Review (20 Punkte)

Der folgende **absichtlich fehlerhafte C-Ausschnitt** soll jeden angenommenen Arbeitshinweis bearbeiten. Ein gültiger Mutex ist bereits angelegt; Boot und GPIO sind geprüft. `HAL_Delay` ist in diesem Projekt aktives Warten auf die HAL-Millisekundenbasis. `osDelay` verwendet Kernelticks. Aussagen über konkrete Interleavings beziehen sich auf das gezeigte Maschinenmodell; dies ist kein Beispiel für korrekte portable C-Synchronisation.

Benennen Sie **vier unterschiedliche Probleme** und schlagen Sie einen passenden Kommunikations-/Wartevertrag vor. Zählen Sie Mutexverwendung und den zusätzlich blockierenden Parameter desselben ISR-Aufrufs nicht als zwei unabhängige Funde. Muss ein unzulässiger API-Aufruf zwangsläufig einen HardFault erzeugen? *(20 Punkte)*

```c
static volatile uint32_t pending = 0;
osMutexId_t dataMutex; /* gültig angelegt */
void HAL_GPIO_EXTI_Callback(uint16_t pin) {
    osMutexAcquire(dataMutex, osWaitForever); /* Ergebnis ignoriert */
    ++pending;
    osMutexRelease(dataMutex);
}
void ProcessTask(void *argument) {
    for (;;) {
        while (pending == 0) { }
        pending = 0;
        Filter_Complex_Data(); /* bearbeitet einen Hinweis */
        HAL_Delay(50);
    }
}
```

Ihre Lösung

(Benennen Sie Ursache, Folge und eine passende Korrektur.)